

Laboratorio 1 - Manejo e Implementación de Archivos

Este documento explica las decisiones técnicas del proyecto: qué formato de archivo se usó y por qué, cómo se implementó la escritura segura y el respaldo, cómo se manejan los errores, y evidencia de que todo funciona.

1. Formato de almacenamiento: JSON

Se guarda la configuración en un archivo JSON (`config.json`). Se comparó contra CSV, que fue la alternativa descartada.

Criterio	JSON (elegido)	CSV (descartado)
Tipos de datos	Soporta texto, números y anidados sin problema	Todo se guarda como texto, hay que convertir tipos
Legibilidad	Fácil de leer, cada campo con su nombre	Depende del orden de las columnas
Detectar corrupción	<code>json.load()</code> lanza un error claro si está mal	Puede "parsear" mal sin avisar (columnas corridas)
Casos de uso típico	Objetos con distintos tipos de datos (nuestro caso)	Tablas con muchas filas de datos iguales

En resumen: nuestra configuración tiene datos de distintos tipos (texto, número, colores) en un solo objeto, no filas repetidas. Para eso JSON es más natural que CSV, y además si el archivo se corrompe, Python lo detecta solo con una excepción (`json.JSONDecodeError`) en vez de leer datos raros sin darse cuenta.

2. Escritura segura y respaldo

Para que un cierre inesperado de la app no deje el archivo a medias, `guardar_config()` nunca escribe directo sobre `config.json`. Hace esto:

- 1) Escribe todo el contenido nuevo en `config.json.tmp`
- 2) Si `config.json` ya existía, lo copia a `config.json.bak` (respaldo)
- 3) Usa `os.replace()` para reemplazar `config.json` por el `.tmp` de un solo golpe

```

def guardar_config(datos, ruta=CONFIG_FILENAME):
    ruta_temporal = ruta + ".tmp"
    ruta_respaldo = ruta + BACKUP_SUFFIX

    try:
        with open(ruta_temporal, "w", encoding="utf-8") as archivo:
            json.dump(datos, archivo, ensure_ascii=False, indent=4)
            archivo.flush()
            os.fsync(archivo.fileno())

        if os.path.exists(ruta):
            _copiar_archivo(ruta, ruta_respaldo)    # respaldo

        os.replace(ruta_temporal, ruta)            # reemplazo atomico
        return True, "Configuracion guardada correctamente."

    except PermissionError:
        return False, "Sin permisos de escritura. No se guardaron los cambios."

```

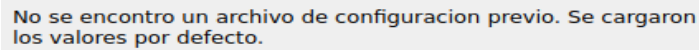
`os.replace()` es atómico: o el archivo final queda completo con los datos nuevos, o se queda tal cual estaba antes. Nunca hay un estado "a medias". La copia a .bak se hace con una función propia (`_copiar_archivo`), usando solo `open()` en modo binario, sin librerías externas de copiado.

3. Manejo de errores

La app maneja 3 casos sin caerse nunca con un traceback. Cada uno degrada a un comportamiento definido:

Caso 1: Archivo ausente

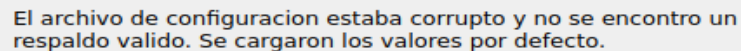
Si config.json no existe (primera vez que se abre la app), se cargan los valores por defecto y se avisa al usuario con un mensaje.



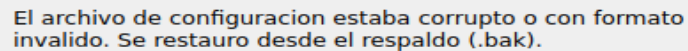
```
No se encontro un archivo de configuracion previo. Se cargaron los valores por defecto.
```

Caso 2: Archivo corrupto

Si config.json tiene un JSON mal formado, primero se intenta recuperar desde config.json.bak. Si el respaldo también falla o no existe, se usan los valores por defecto.



```
El archivo de configuracion estaba corrupto y no se encontro un respaldo valido. Se cargaron los valores por defecto.
```



```
El archivo de configuracion estaba corrupto o con formato invalido. Se restaura desde el respaldo (.bak).
```

Caso 3: Sin permisos de lectura

Esta prueba no se puede hacer como usuario root (root ignora los permisos de archivo), así que se probó con un usuario Linux normal. Resultado real obtenido:

```
$ chmod 000 config.json
$ su testuser -c "python3 -c 'from config_manager import cargar_config; ...'"
```

```
ESTADO: sin_permisos
MENSAJE: No se tienen permisos de lectura sobre el archivo de configuracion.
        Se cargaron los valores por defecto.
```

cargar_config() captura PermissionError de forma explícita y no deja que la excepción tumbe la aplicación.

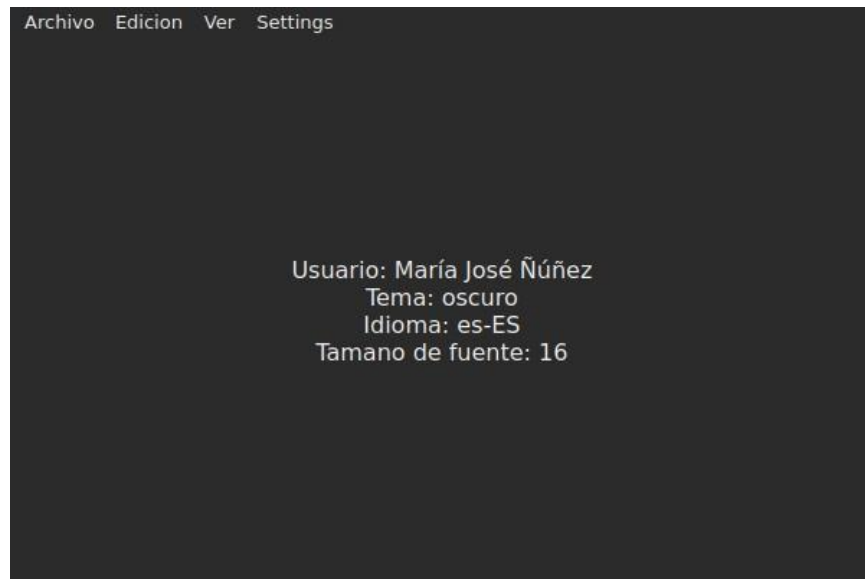
4. Ejemplo de archivo generado

Config.json real generado por la app, con tildes y ñ para comprobar que la codificación UTF-8 no corrompe los caracteres especiales:

```
{
  "nombre_usuario": "María José Núñez",
  "tema_interfaz": "oscuro",
  "idioma": "es-ES",
  "tamano_fuente": 16,
  "color_barra_menu": "#2b2b2b",
  "color_letra": "#eeeeee",
  "foto_perfil": ""
}
```

5. Capturas del funcionamiento

Ventana principal con la configuración cargada:



Tema oscuro, colores y tamaño de fuente aplicados desde config.json

Ventana de Settings:

Nombre de usuario:	María José Núñez
Tema de interfaz:	oscuro
Idioma:	es-ES
Tamano de fuente:	16
Color de la barra de menu:	Elegir color... 
Color de letra:	Elegir color... 
Foto de perfil:	Seleccionar imagen... (invalida)